1. Scope and Requirements

1.1 System Purpose

The purpose of the application is to provide a small booking management system in which users can access and process their own booking information while privileged administrators can perform restricted management operations.

The application is designed as a multi-tier system consisting of three separate Python services communicating through HTTP APIs. The separation of responsibilities is intended to make authorization decisions and trust relationships between components explicit.

The system uses only synthetic data and seeded user accounts.

1.2 Users and Roles

The system has two roles:

User
An ordinary user who can access functionality related to resources they are authorized to use.

Administrator
A privileged user who can perform restricted administrative operations in addition to normal authorized functionality.

Three accounts will be seeded:

- John – User
- Doe – User
- Admin – Administrator

Having two ordinary users allows authorization tests to verify that one user cannot access another user's protected resources.

1.3 Application Components

The application consists of three independently running components:

API Service
Authenticates users, establishes their identity and coordinates requests to the other services.

Resource Service

Owns the protected booking records and enforces authorization rules for operations on those records.

Processing Service

Performs booking-processing operations and uses the project's separately installable helper package.

Communication between the components will take place through explicit HTTP interfaces.

1.4 Protected Resource

The primary protected resource is booking.

Each booking is associated with an owner. Authorization controls determine whether a requesting user or service is permitted to perform an operation on that booking.

1.5 Functional Requirements

FR1 – Authentication

The system shall authenticate a user before allowing access to protected functionality.

FR2 – View Own Bookings

An ordinary user shall be able to retrieve bookings that they are authorized to access.

FR3 – Booking Administration

An administrator shall be able to perform a restricted booking-management operation.

FR4 – Booking Summary

An authenticated user shall be able to request a summary of their authorized booking information. This workflow shall involve the API Service, Resource Service and Processing Service.

FR5 – Authorization Enforcement

The system shall enforce authorization rules so that ordinary users cannot access another user's protected booking resources.

FR6 – External Helper Package

The Processing Service shall use a small separately packaged and installable helper dependency when performing its processing functionality.

1.6 Non-Functional and Security Requirements

NFR1 – Component Separation

The three application components shall run as separate processes.

NFR2 – Explicit Communication

Communication between components shall occur through explicit network interfaces.

NFR3 – Least Privilege

Components and users shall receive only the permissions required to perform their intended operations.

NFR4 – Complete Mediation

Access to protected resources shall be checked when protected operations are requested and shall not rely solely on an earlier authorization decision made by another component.

NFR5 – Fail-Safe Defaults

Requests for which sufficient authorization cannot be established shall be denied by default.

NFR6 – Testability

The application shall support automated functional and authorization testing.

NFR7 – Reproducible Build

Application dependencies and the helper package shall be declared so that the application environment can be reproduced.

1.7 Out of Scope

To keep the application focused on the security objectives of the assignment, the following are outside the project scope:

- User registration
- Graphical user interface
- Real payments
- Email delivery
- Public hosting

- High availability
- Large-scale performance and scalability
- Custom cryptographic algorithms
- Optional controlled supply-chain attack experiments

1.8 Planned Vulnerability Investigation

As part of the security analysis, the application will intentionally contain four controlled vulnerabilities. Three vulnerabilities are selected from the course vulnerability catalogue, while one additional vulnerability will arise from the composition and trust relationships between the application's components.

The vulnerabilities will be introduced in a controlled development environment for analysis and testing. Their security assumptions, exploitability and eventual mitigation will be documented.

V1 – CWE-862: Missing Authorization

A controlled missing-authorization vulnerability will be introduced around access to protected booking resources.

The vulnerable implementation will demonstrate the difference between authentication and authorization. A user may be successfully authenticated while the application fails to adequately verify whether that user owns, or is otherwise permitted to access, the requested booking.

This vulnerability will be used to demonstrate ownership-based authorization and the importance of enforcing authorization at the protected resource boundary.

V2 – CWE-89: SQL Injection

A controlled SQL injection vulnerability will be introduced in an operation involving booking data.

The vulnerable implementation will demonstrate how externally influenced data can become part of an SQL command when command structure and data are not adequately separated.

The vulnerability will later be compared with a secure implementation using parameterized database operations.

V3 – CWE-20: Improper Input Validation

A controlled input-validation vulnerability will be introduced at an application interface.

The vulnerable implementation will accept data without adequately establishing that it satisfies the format, type, range, structure or semantic constraints required by the subsequent operation.

The vulnerability will demonstrate why data crossing a trust boundary must be validated before it is used in security-relevant processing.

V4 – Stale Authorization Between Services

The fourth vulnerability will arise from the composition of the application's independently running services.

In the vulnerable workflow, the API Service performs an authorization check before sending a processing request to another service. The Processing Service subsequently performs the requested operation based on this earlier authorization decision.

The vulnerable design will not adequately verify whether the user's authorization is still valid when the protected resource is accessed. If the user's permission changes or is revoked after the initial authorization decision, another component may still complete the previously authorized operation.

This demonstrates how authorization decisions can become stale when authority is transferred between independently running components and reused without revalidation.

The secure implementation will ensure that authorization is verified at the protected resource boundary when the operation is performed rather than relying only on an earlier authorization decision made by another component.

Vulnerability Handling

The project will investigate four intentionally introduced vulnerabilities:

1. **CWE-862 – Missing Authorization**
2. **CWE-89 – SQL Injection**
3. **CWE-20 – Improper Input Validation**
4. **Stale Authorization Between Services – Composition Vulnerability**

For each vulnerability, the project will document the violated security assumption, vulnerable behaviour, conditions required to demonstrate the vulnerability, security impact, and implemented mitigation.

Where appropriate, automated tests will provide evidence of both the vulnerable behaviour and the corrected implementation.

All vulnerability demonstrations will be limited to the local application, seeded accounts, and synthetic booking data created for this assignment.

2. Operation List

The following operations define the main functionality of the Booking Management System and identify the components involved in each operation.

ID	Operation	Actor	Components Involved	Description
OP1	Login	User / Admin	API Service	Authenticates a seeded user and establishes the user's identity and role.
OP2	View Own Bookings	User / Admin	API Service, Resource Service	Retrieves booking records that the authenticated user is authorized to access.
OP3	View Specific Booking	User / Admin	API Service, Resource Service	Retrieves a specific booking after verifying that the requester is authorized to access it.
OP4	Manage Booking	Admin	API Service, Resource Service	Allows an administrator to perform a restricted management operation on a booking.
OP5	Generate Booking Summary	User / Admin	API Service, Processing Service, Resource Service	Requests processing of authorized booking data. The API Service coordinates the request, the Processing Service performs the summary operation using the helper package, and the Resource Service provides the protected booking data.
OP6	Retrieve Booking Summary	User / Admin	API Service, Resource Service	Retrieves a previously generated booking summary if the requester is authorized to access it.

2.1 Three-Component Workflow

OP5 is the application's primary multi-component workflow and involves all three application components.

The workflow is:

User → API Service → Processing Service → Resource Service → Processing Service → API Service → User

The API Service authenticates the requester and initiates the processing operation. The Processing Service requests the required booking information from the Resource Service and uses the project's helper package to generate the summary.

Authorization must be maintained throughout this workflow. In particular, the Resource Service must enforce access to the protected booking information rather than assuming that a request is authorized simply because it originated from another application component.

3. Asset Register

The asset register identifies information and resources that require protection within the Booking Management System.

ID	Asset	Owner / Responsible Component	Security Requirement	Description
A1	User Credentials	API Service	Confidentiality, Integrity	Credentials belonging to the seeded John, Doe and Admin accounts. They must not be exposed or modified by unauthorized users.
A2	User Identity and Role	API Service	Integrity, Authenticity	The authenticated identity and assigned role used when making authorization decisions.
A3	Authentication Token / Session	API Service	Confidentiality, Integrity	Security information used to associate requests with an authenticated user and their authority.
A4	Booking Records	Resource Service	Confidentiality, Integrity	Protected booking information belonging to individual users. Users must not access or modify another user's bookings unless explicitly authorized.
A5	Booking Ownership Information	Resource Service	Integrity	Information connecting a booking to its owner. Incorrect ownership information could lead to unauthorized access.

A6	Booking Summaries	Resource Service	Confidentiality, Integrity	Generated results derived from protected booking information. Access to a summary must follow the authorization rules of the underlying booking data.
A7	Inter-Service Requests	API, Resource and Processing Services	Integrity, Authenticity	Requests exchanged between components containing security-relevant information such as user identity, booking identifiers and requested operations.
A8	Helper Package	Processing Service	Integrity	The separately installable dependency used to generate booking summaries. The application must use the expected dependency rather than an unintended or modified package.
A9	Security Evidence and Test Results	Project / CI Pipeline	Integrity, Availability	Test results, static-analysis results and other evidence used to demonstrate that security requirements and release gates have been evaluated.

3.1 Primary Protected Assets

The primary protected assets are the booking records and generated booking summaries.

Booking records are owned by individual users. The Resource Service is responsible for enforcing access to these records and their derived results.

Identity, role and authentication information are also security-critical because incorrect or manipulated authority information could cause the Resource Service or Processing Service to perform an operation that the requester is not permitted to perform.

4. Architecture and Trust Boundaries

4.1 Architecture Overview

The Booking Management System uses a multi-tier architecture consisting of three independently running Python services. The components communicate through explicit HTTP interfaces.

The components have separate responsibilities so that authentication, protected-resource ownership and processing are not handled by a single component.

API Service

The API Service is the entry point to the application.

Responsibilities:

- Authenticate users.
- Establish user identity and role.
- Accept user requests.
- Coordinate operations with the Resource Service and Processing Service.
- Return results to the user.

The API Service does not directly own the protected booking records.

Resource Service

The Resource Service owns the protected booking records and generated booking summaries.

Responsibilities:

- Store and retrieve booking information.
- Maintain booking ownership information.
- Enforce authorization when protected resources are accessed.
- Store and retrieve generated booking summaries.

The Resource Service acts as the main security boundary protecting user-owned resources.

Processing Service

The Processing Service performs booking-processing operations.

Responsibilities:

- Receive authorized processing requests.
- Request required booking information from the Resource Service.
- Generate booking summaries.
- Use the separately installable helper package during summary generation.

The Processing Service should only operate on information required for the specific processing request.

4.2 Trust Boundaries

A trust boundary exists whenever information moves between components with different responsibilities or levels of authority.

TB1 – User → API Service

User-controlled input crosses into the application. Authentication credentials, booking identifiers and other request data must not automatically be considered trusted.

TB2 – API Service → Resource Service

Requests containing identity, resource and operation information cross from the coordinating service to the service that owns protected resources. The Resource Service must enforce its own authorization requirements before allowing access.

TB3 – API Service → Processing Service

Processing requests cross into a separately running service. The Processing Service must not assume that all data is safe merely because it originated from the API Service.

TB4 – Processing Service → Resource Service

The Processing Service requests protected booking information during processing operations. The Resource Service must ensure that the request is within the authority associated with the operation.

This boundary is particularly important when analysing authorization decisions that may become stale between services.

TB5 – Processing Service → Helper Package

Booking information is supplied to an externally packaged dependency. Only the information required for the processing operation should be provided to the helper package.

4.3 Security Responsibility

Authentication is primarily established by the API Service, while authorization over protected booking resources is ultimately enforced by the Resource Service.

This separation prevents successful authentication at the API Service from automatically granting unrestricted access to resources.

The Processing Service receives only the information and authority required for the processing operation. Security-relevant information crossing service boundaries must not be trusted solely because it originated from another application component.

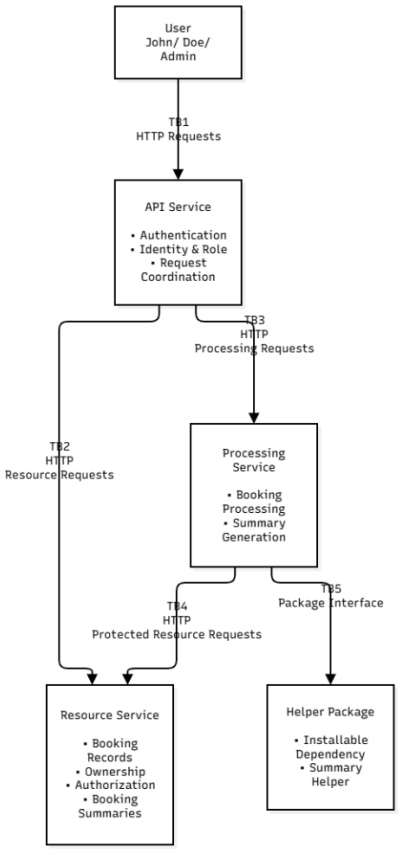


Figure 1 – System Architecture and Trust Boundaries

5. Access-Control Matrix

The application uses role-based and ownership-based authorization.

Ordinary users may access resources they own but must not access resources belonging to another ordinary user. The Administrator role is permitted to perform restricted management operations.

Authorization over protected booking resources is enforced by the Resource Service.

Operation	**John – User**	**Doe – User**	**Admin**	**Processing Service**
Login	Allow	Allow	Allow	N/A
View John's bookings	Allow	Deny	Allow	Only for an authorized processing job
View Doe's bookings	Deny	Allow	Allow	Only for an authorized processing job
View own specific booking	Allow	Allow	Allow	Only for an authorized processing job
View another user's booking	Deny	Deny	Allow	Only if permitted by the job's authority
Manage own booking	Deny	Deny	Allow	Deny
Manage another user's booking	Deny	Deny	Allow	Deny
Generate summary of own booking	Allow	Allow	Allow	Execute authorized job
Generate summary of another user's booking	Deny	Deny	Allow	Only if permitted by the job's authority
Retrieve own generated summary	Allow	Allow	Allow	N/A
Retrieve another user's summary	Deny	Deny	Allow	N/A

5.1 Authorization Rules

The following authorization rules are derived from the access-control matrix:

AC1 – Authentication Required
Protected operations require an authenticated identity.

AC2 – Ownership Enforcement
An ordinary user may access only booking records and summaries belonging to that user.

AC3 – Administrative Authority
The Administrator may access booking records belonging to ordinary users and perform restricted booking-management operations.

AC4 – Processing-Service Restriction
The Processing Service does not receive unrestricted access to all booking data. Access should be limited to resources required for an authorized processing job.

AC5 – Resource-Service Enforcement
The Resource Service must enforce authorization when protected booking information is accessed, including requests originating from another application service.

AC6 – Default Denial
If the system cannot establish that an operation is permitted, access is denied.

6. STRIDE Threat Analysis

The STRIDE model is used to identify security threats affecting the Booking Management System, its protected resources, and the trust boundaries between components.

STRIDE Category	Example Threat in the System	Affected Asset / Boundary	Planned Control
Spoofing	An attacker attempts to act as John, Doe, or Admin using invalid or manipulated authentication information.	User identity, authentication token, TB1	Authentication is required and identity information is validated by the API Service.
Tampering	A requester modifies a booking identifier or other security-relevant request data in an attempt to access or alter an unintended booking.	Booking records, inter-service requests, TB1/TB2	Validate input and enforce authorization against the requested resource.
Repudiation	A user performs a security-relevant booking operation and later denies performing it.	Booking operations and security evidence	Record relevant security events containing the authenticated identity, operation, resource and result.
Information Disclosure	John attempts to retrieve Doe's booking information or generated summary.	Booking records and summaries, TB2/TB4	Ownership checks and authorization are enforced by the Resource Service.
Denial of Service	A client repeatedly submits malformed or excessive processing requests that consume service resources.	API Service and Processing Service	Input constraints, controlled request handling and rejection of invalid requests.
Elevation of Privilege	An ordinary user attempts to perform an administrator-only booking-management operation.	User role, booking records	Role-based authorization and default denial of unauthorized operations.

6.1 Threats Across Component Boundaries

The separation of the application into independently running services creates additional trust relationships.

The API Service establishes the user's identity, but downstream services must not assume that every received request is automatically authorized. In particular, the Resource Service owns the protected booking information and is responsible for enforcing access to that information.

The Processing Service is also restricted to the authority required for a specific processing job rather than receiving unrestricted access to all booking resources.

Authorization information that crosses service boundaries must remain valid for the operation being performed. Reusing an earlier authorization decision after the user's authority has changed can create a stale-authorization vulnerability.

6.2 Relationship to Planned Vulnerabilities

The planned vulnerabilities are related to threats identified through the STRIDE analysis:

- **CWE-862 – Missing Authorization:** primarily Information Disclosure and Elevation of Privilege.
- **CWE-89 – SQL Injection:** primarily Tampering and potentially Information Disclosure.
- **CWE-20 – Improper Input Validation:** may enable Tampering or other unintended behaviour when invalid input reaches sensitive operations.
- **Stale Authorization Between Services:** may result in Information Disclosure or Elevation of Privilege when an earlier authorization decision remains trusted after it is no longer valid.

The intentionally vulnerable versions will be used to demonstrate these security failures. Corrected versions will introduce or restore the required controls and will be verified through security tests.

7. Secure Design Principles

The Booking Management System applies three secure design principles: least privilege, complete mediation, and fail-safe defaults.

These principles influence the separation of responsibilities between components and the authorization controls protecting booking resources.

7.1 Least Privilege

Design Decision

Users and application components should receive only the permissions required to perform their intended operations.

Ordinary users are restricted to their own booking resources, while administrators receive additional permissions for restricted management operations.

The Processing Service should not receive unrestricted access to all booking records. It should access only the data required for an authorized processing job.

Planned Implementation

- John and Doe can access only their own protected resources.
- Administrator privileges are restricted to the Admin role.
- The Processing Service operates only on booking data required for a specific processing request.
- Component responsibilities are separated between authentication, resource ownership, and processing.

Planned Test / Evidence

Authorization tests will verify that:

- John cannot access Doe's protected bookings.
- Doe cannot access John's protected bookings.
- Ordinary users cannot perform administrator-only operations.
- Processing operations cannot access resources outside their authorized scope.

7.2 Complete Mediation

Design Decision

Authorization must be checked when a protected resource is accessed rather than relying entirely on an earlier decision made by another component.

This is especially important because the application consists of multiple independently running services.

Planned Implementation

The Resource Service will enforce authorization when protected booking records and summaries are requested.

Requests originating from the API Service or Processing Service will not automatically be considered authorized merely because they originate from another trusted application component.

Security-relevant information such as identity, requested operation, resource ownership and current authority will be considered when access is granted.

Planned Test / Evidence

Tests will verify that:

- Requests for another user's resources are rejected.
- Direct inter-service requests cannot bypass the required authorization controls.
- Authorization is checked when the protected resource is accessed.
- An outdated authorization decision cannot be reused after the relevant authority is no longer valid.

7.3 Fail-Safe Defaults

Design Decision

Access will be denied unless the application can establish that the requested operation is permitted.

Missing, invalid, malformed or insufficient authorization information must not result in access being granted.

Planned Implementation

Protected endpoints will require authentication and authorization information.

If the Resource Service cannot establish that a requester is permitted to access a booking, the request will be rejected.

Administrator functionality will require explicit administrator authority rather than treating unspecified roles as privileged.

Planned Test / Evidence

Tests will verify that:

- Unauthenticated requests to protected operations are rejected.
- Invalid authentication information is rejected.
- Requests with insufficient permissions are rejected.
- Unknown or invalid roles do not receive privileged access.
- Missing security-relevant information results in denial rather than access.

8. Implementation and Build Plan

8.1 Implementation Approach

The Booking Management System will be implemented in Python as three independently running application components:

1. API Service
2. Resource Service
3. Processing Service

Each service will run as a separate process and communicate through HTTP interfaces.

Development will be performed incrementally, with Git commits used to preserve the implementation history and security-relevant changes.

8.2 Component Implementation

API Service

The API Service will implement authentication, establish user identity and role, expose the main application interface, and coordinate requests to the Resource Service and Processing Service.

Resource Service

The Resource Service will own booking records, booking ownership information, and generated booking summaries.

Authorization controls will be implemented at this component because it owns the protected resources.

Processing Service

The Processing Service will perform the booking-summary workflow. It will request authorized booking information from the Resource Service and process the information using the separately installable helper package.

8.3 External Helper Package

A small Python helper package will be developed separately from the three application components.

The package will provide functionality used during booking-summary generation and will be declared and installed as a project dependency rather than copied directly into the Processing Service source code.

8.4 Dependencies and Reproducible Builds

Python dependencies will be explicitly declared in project dependency files.

The project repository will contain the information required to reproduce the development and test environment.

Unexpected or undeclared dependencies should cause the relevant build or validation gate to fail.

8.5 Vulnerability Implementation

The project will intentionally introduce controlled implementations of the following vulnerabilities for security analysis:

- CWE-862 – Missing Authorization
- CWE-89 – SQL Injection
- CWE-20 – Improper Input Validation
- Stale Authorization Between Services – Composition Vulnerability

The vulnerable behaviour will be demonstrated only using the project's synthetic users and booking data.

Evidence of the vulnerable behaviour and subsequent mitigation will be preserved through source-code history, tests, and relevant analysis results.

The final implementation will enforce the security guarantees defined in the requirements and access-control matrix.

8.6 Implementation Gate

Before implementation is considered complete:

- All three application components must run independently.
- Required dependencies must be explicitly declared.
- The helper package must be installable as a dependency.
- The required application operations must function across the intended component boundaries.
- Unexpected or unapproved dependencies must be rejected.

9. Testing and CI/CD Plan

9.1 Testing Approach

Automated testing will be used to verify both application functionality and the security requirements defined in the lifecycle plan.

The test suite will include functional tests, authentication tests, authorization tests, input-validation tests, and tests covering communication between the three application components.

Testing will use only the seeded John, Doe and Admin accounts and synthetic booking data.

9.2 Functional Testing

Functional tests will verify that the required application operations behave correctly.

Tests will include:

- Successful authentication using seeded accounts.
- Retrieval of a user's own bookings.
- Administrator booking-management functionality.
- Generation of a booking summary.
- Successful execution of the workflow involving all three application components.
- Retrieval of an authorized generated summary.

9.3 Authorization Testing

Authorization tests will verify the access-control matrix.

Examples include:

- John can access John's bookings.
- John cannot access Doe's bookings.
- Doe can access Doe's bookings.
- Doe cannot access John's bookings.
- Ordinary users cannot perform administrator-only operations.
- Admin can perform the permitted administrative operations.
- Unauthorized processing requests cannot access protected booking information.
- Missing or invalid authority results in access being denied.

9.4 Vulnerability Testing

Controlled tests will be used to demonstrate and investigate the intentionally introduced vulnerabilities:

- CWE-862 – Missing Authorization
- CWE-89 – SQL Injection
- CWE-20 – Improper Input Validation
- Stale Authorization Between Services – Composition Vulnerability

Evidence of the vulnerable behaviour will be preserved where appropriate.

After investigation, corrected implementations will be tested to verify that the required security guarantees are enforced.

9.5 Static and Dependency Analysis

Automated static analysis will be used to identify relevant security or code-quality findings.

Dependency analysis will be used to identify known security issues in project dependencies.

Analysis results will be produced during the CI/CD pipeline and preserved as build artifacts where appropriate.

9.6 Jenkins Pipeline

A Jenkins pipeline will execute on every push and pull request.

The pipeline will:

1. Validate or build the three application components and helper package.
2. Install the explicitly declared dependencies.
3. Run static analysis.
4. Run dependency analysis.
5. Start a fresh test environment containing the three application services.
6. Run functional tests.
7. Run authentication and authorization tests.
8. Save relevant test and analysis results as pipeline artifacts.

9.7 Release-Blocking Conditions

The CI/CD pipeline will fail when a confirmed finding violates a required security guarantee.

Release-blocking conditions include:

- A required application component fails to start.
- Required functional tests fail.
- Authentication tests fail.
- Authorization tests show that an ordinary user can access another user's protected resource.

- An ordinary user can perform an administrator-only operation.
- The three-component workflow fails.
- A confirmed high-severity dependency or static-analysis finding violates a required security guarantee.
- An unexpected or unapproved dependency is introduced.

Confirmed violations will be investigated and corrected before the build is considered suitable for release.

9.8 Test Evidence

Test reports and analysis results generated by the pipeline will be retained as artifacts.

These artifacts, together with source-code commits and Git history, will provide evidence connecting security requirements and design decisions to their implementation and verification.

10. Release, Deployment and Operation Plan

10.1 Assignment Environment

The Booking Management System is developed as a local educational application using synthetic users and booking data.

Public deployment is outside the scope of the assignment. The application will therefore be executed locally or in an isolated test environment for development, security analysis and automated testing.

10.2 Release Requirements

Before a version of the application is considered suitable for release, the following conditions must be satisfied:

- All three application components start and communicate correctly.
- Required functional tests pass.
- Authentication and authorization tests pass.
- The access-control rules defined in the lifecycle plan are enforced.
- Confirmed security findings that violate required security guarantees are corrected.
- Required dependencies are explicitly declared.
- Static and dependency analysis have completed successfully according to the defined release gates.
- Required test and analysis artifacts have been generated.

A failed release-blocking condition in the Jenkins pipeline prevents the version from being considered releasable.

10.3 Production Deployment Considerations

Although production deployment is outside the scope of this assignment, a real deployment would require additional security controls.

These would include:

- HTTPS/TLS for communication between users and services.
- Secure management of credentials, tokens and other secrets.
- Stronger production authentication and session management.
- Network restrictions between application components.
- Secure configuration of the services and their runtime environments.
- Appropriate database access controls and protection of stored data.
- Centralized security logging and monitoring.

- Dependency and security update procedures.
- Backup and recovery procedures.
- Controlled deployment and rollback procedures.

10.4 Operational Security

In a production environment, logs and monitoring should be used to identify authentication failures, authorization failures and unexpected application behaviour.

Dependencies should be periodically reviewed for known vulnerabilities, and security updates should be tested before deployment.

Changes affecting authentication, authorization, component boundaries or protected resources should require renewed security testing.

10.5 Final Evidence

The final project repository will preserve evidence used to evaluate the security of the application, including:

- Git commit history.
- Source code for all three components.
- Helper package.
- Functional and authorization tests.
- Jenkins pipeline definition.
- Static-analysis and dependency-analysis results.
- Test results and relevant CI/CD artifacts.
- Architecture and threat-model documentation.
- Access-control matrix.
- Initial security lifecycle plan.

The final report will reference relevant code, commits, tests, logs and gate results when supporting security claims.